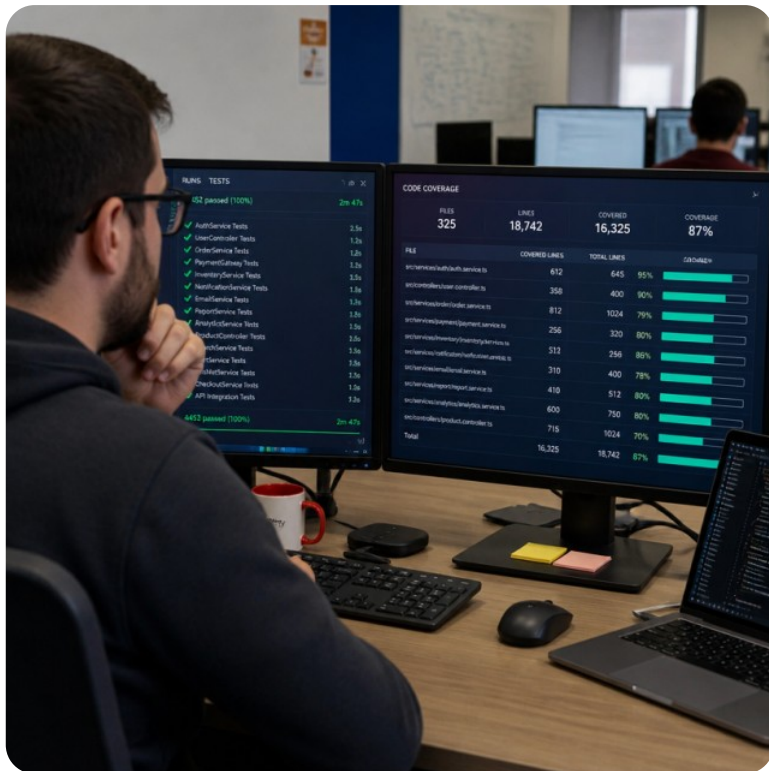


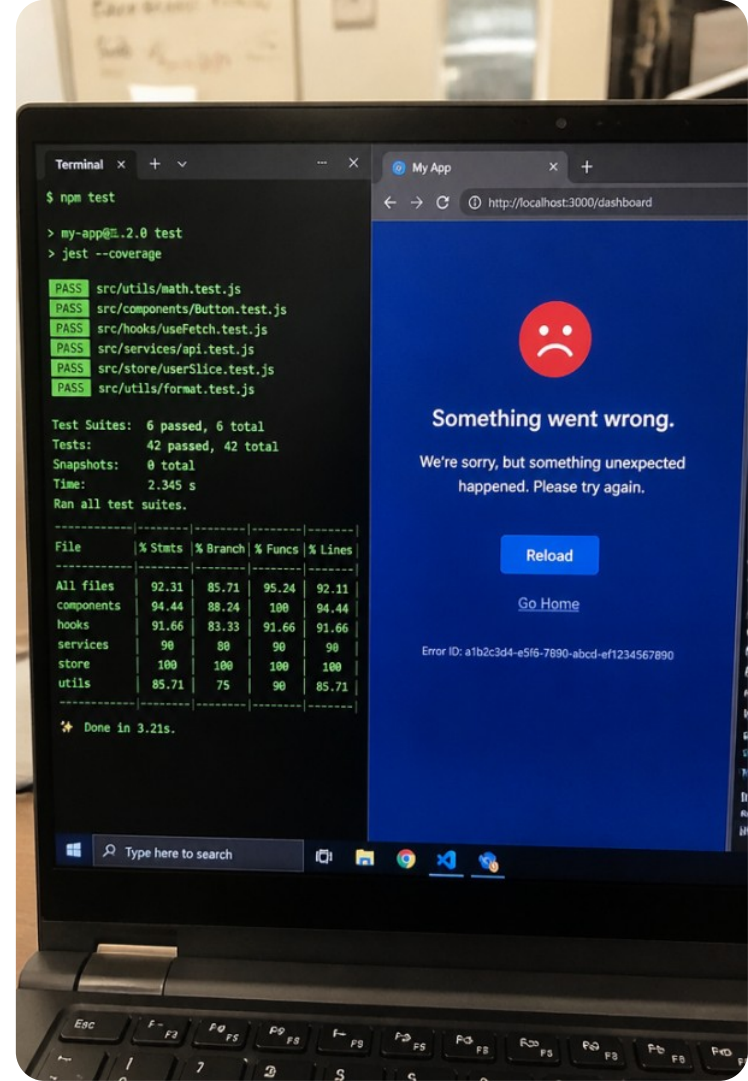


Métricas e Cobertura de Testes

Análise de Code Coverage no Frontend com Vitest e Relatórios

100% de Testes Salva o Sistema?

Um sistema com 100% das linhas executadas por testes pode quebrar no primeiro clique de um usuário real em produção?



Objetivos da Aula

O Que Vamos Construir Hoje

Dominaremos métricas de cobertura:

Diferenciar as 4 métricas estruturais.

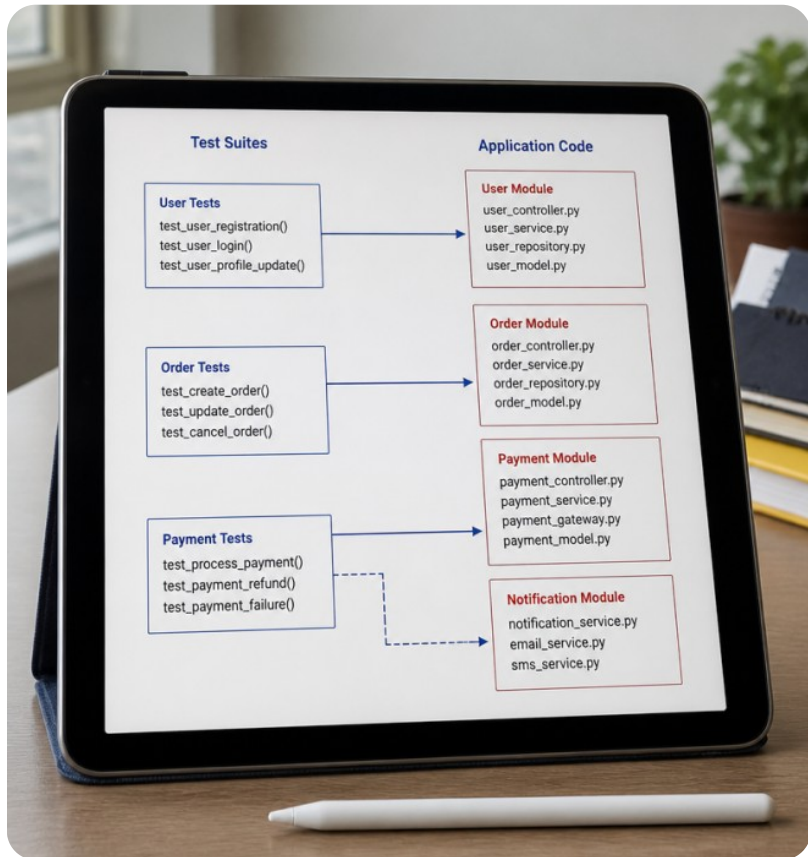
Configurar e ler relatórios HTML no Vitest.

Planejar testes para eliminar trechos cegos.

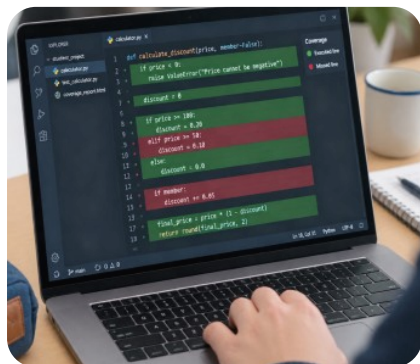


Lembre-se

Cobertura alta indica linhas executadas, não ausência de falhas de lógica.



Vocabulário Essencial



Cobertura:
Percentual de código-fonte exercitado durante a execução dos testes automatizados.



Métrica: Medida quantitativa que avalia a extensão e a profundidade da suíte de testes.

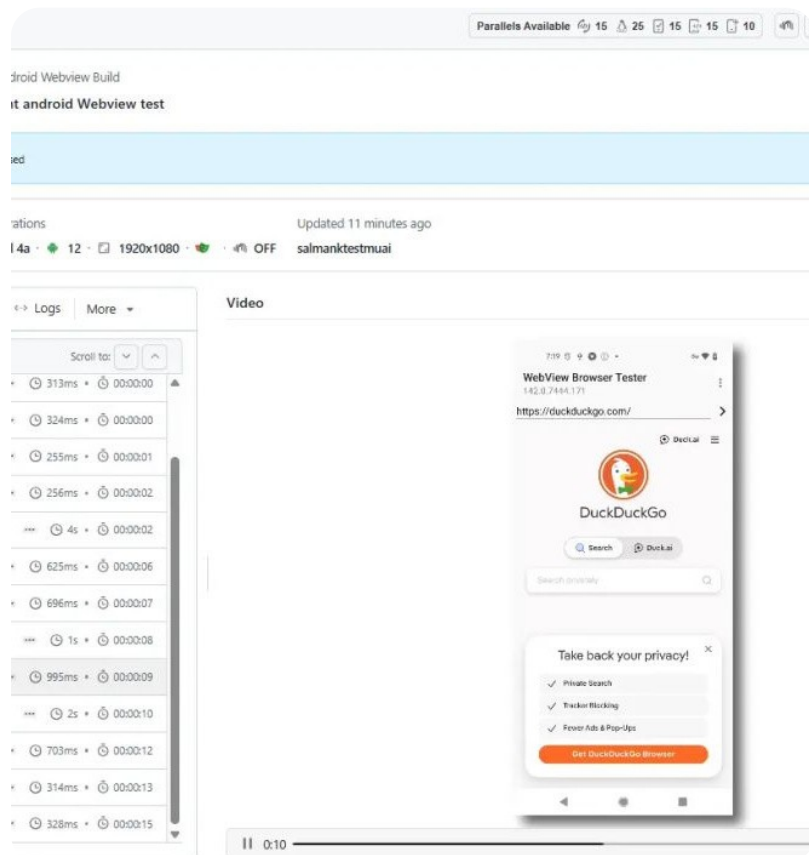


Relatório:
Documento interativo gerado por ferramentas compilando dados e arquivos analisados.



Mitigação: Ações técnicas para reduzir ou eliminar riscos de falhas e bugs em produção.

Recapitulação: Automação E2E



A Camada de Ponta a Ponta

Na aula anterior, exploramos **Cypress** e **Playwright** para validar fluxos reais de ponta a ponta:

Seleção semântica: priorizamos roles de acessibilidade e atributos dedicados.

Interações reais: cliques, navegação e digitação com renderização completa de navegador.

Custo e velocidade: testes poderosos, porém mais lentos que testes unitários.

Mitigação de Flakiness

Estabilidade na Suíte

Testes intermitentes (*flaky tests*) falham sem mudanças no código:

Esperas: use espera de elementos, não timeouts fixos.

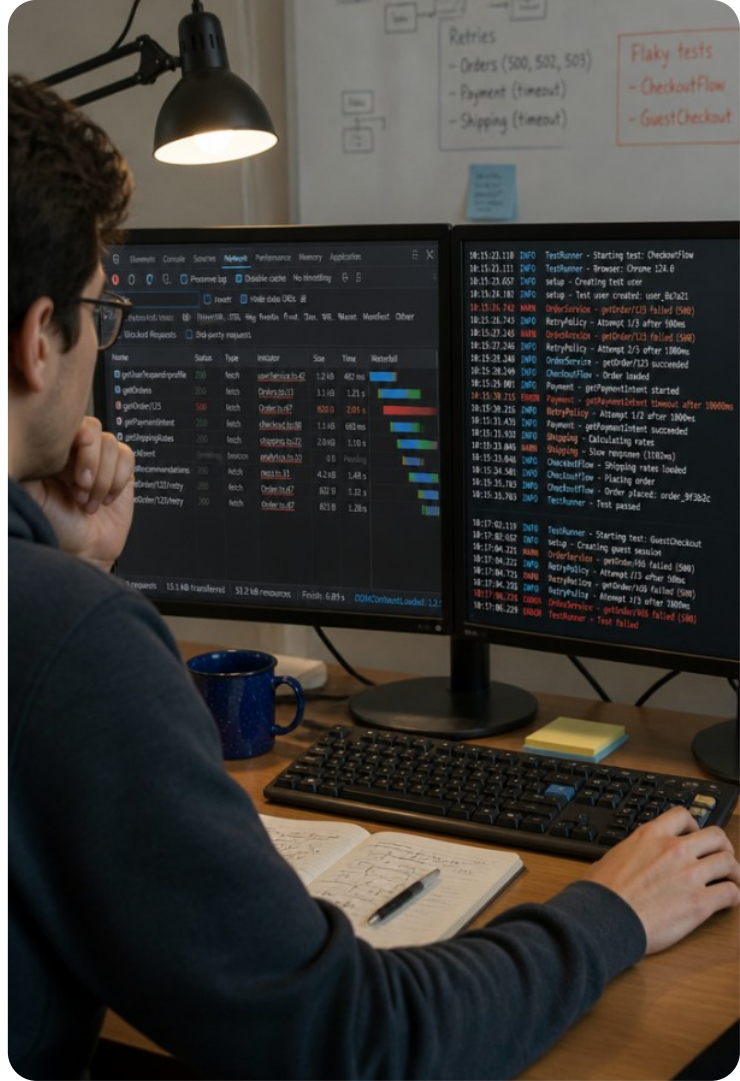
Isolamento: limpe cache, cookies e banco antes de cada teste.

Idempotência: garanta que testes não dependam de dados prévios.



Atenção

Testes instáveis destroem a confiança da equipe nas esteiras de CI.



Quiz: Integração vs E2E



Pergunta 1:

Qual é a principal vantagem dos testes de integração sobre os testes E2E?

Pergunta 2:

Por que mocks de API são comuns em testes de integração mas evitados em E2E rigoroso?

Pergunta 3:

Qual prática mitiga testes E2E intermitentes sem usar timeouts arbitrários?

Quiz: Integração vs E2E



Resposta 1:

Executam muito mais rápido e não dependem do carregamento completo de um navegador real.

Resposta 2:

Porque o teste E2E visa validar o sistema conectado aos serviços reais, enquanto a integração valida componentes com contratos controlados.

Resposta 3:

Aguardar assincronamente por seletores ou eventos específicos da interface via assertions que esperam o estado.



O Ponto Cego do Código

Como saber quais linhas de código da sua aplicação nunca foram executadas por nenhum teste?

O Conceito de Code Coverage

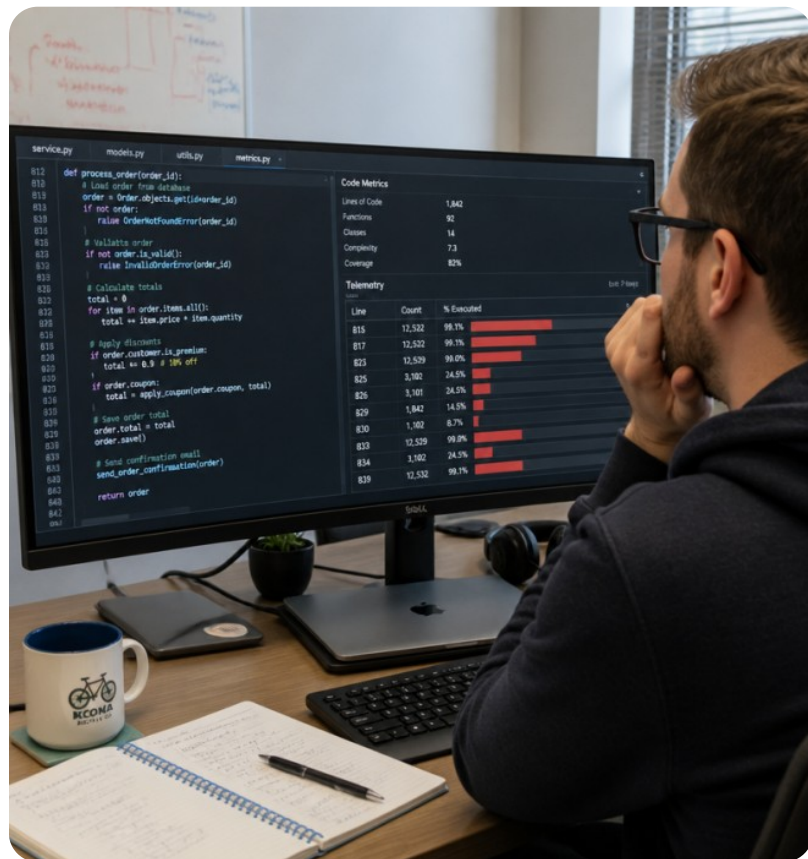
O que é Cobertura?

A **cobertura de código** mede a proporção do código-fonte exercitada pelos testes:

Auditoria de execução: identifica trechos esquecidos pelos desenvolvedores.

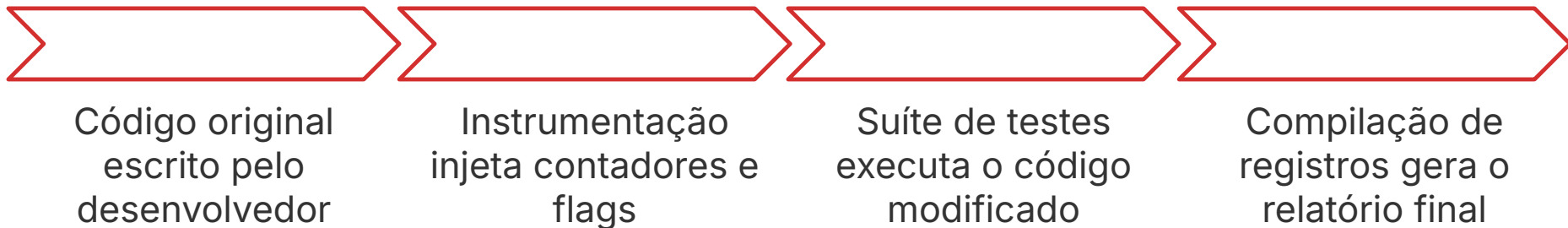
Instrumentação: o compilador injeta contadores invisíveis em cada nó da árvore sintática.

Indicador quantitativo: aponta onde faltam testes, mas não atesta a qualidade das asserções.



Instrumentação de Código

O gerador de cobertura transforma o código antes da execução para registrar as passagens dos testes em memória:



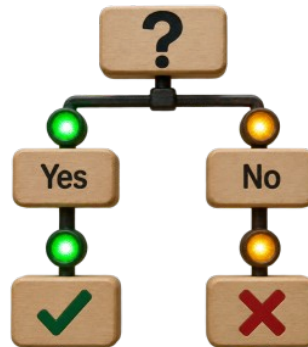
As Quatro Métricas Principais



Statement Coverage:
Percentual de instruções executadas no programa.

```
12 ✓ import json
13 ✓ from pathlib import Path
14
15 ✓ def load_data(file_path):
16 ✓     with open(file_path, "r", encoding="utf-8") as f:
17 ✓         return json.load(f)
18
19 ✓ def process_items(items):
20 ✓     results = []
21 ✓     for item in items:
22 ✓         if item.get('active', False):
23 ✓             results.append(item['name'])
24 ✓     return results
25
```

Line Coverage:
Percentual de linhas físicas tocadas pelo fluxo de teste.

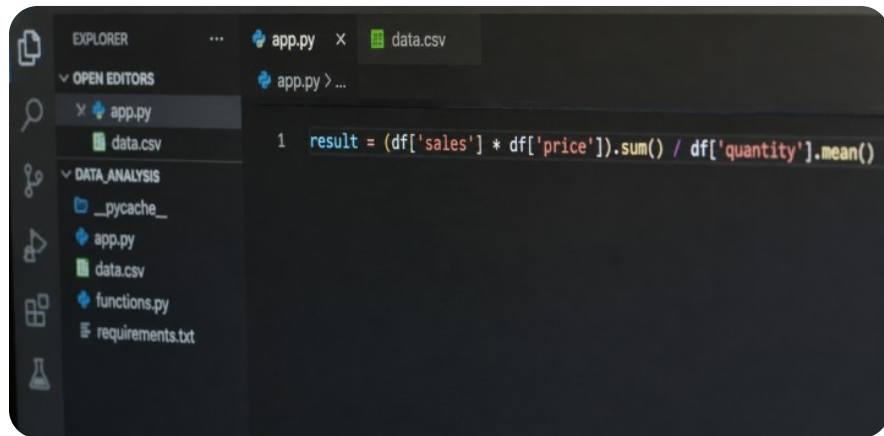


Branch Coverage:
Percentual de caminhos condicionais (if/else, switch) testados.



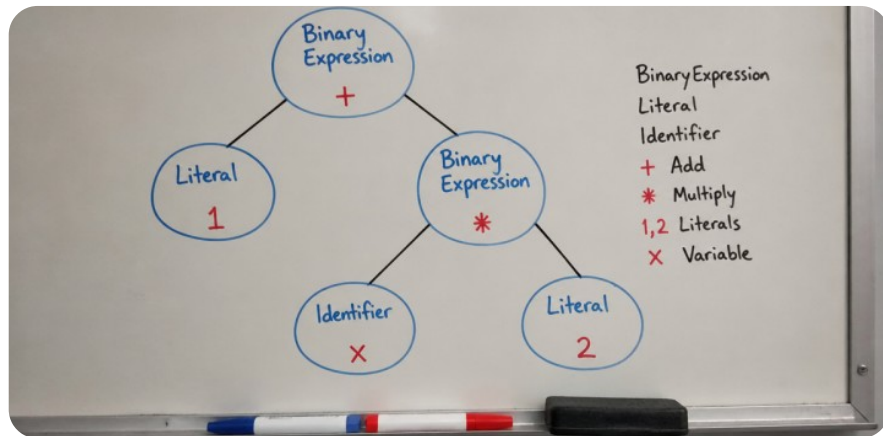
Function Coverage:
Percentual de funções e métodos chamados na suíte.

Statement vs Line Coverage



Linha Física

Uma linha pode conter múltiplos comandos. Se o teste executa apenas a primeira parte de , a linha é marcada como visitada, mascarando lacunas.



Instrução Isolada

Statement coverage analisa nós individuais da árvore sintática (AST). Garante que cada declaração seja computada independentemente da formatação.

A Importância do Branch Coverage

Onde os Bugs Moram

Branch Coverage é vital para lógica:

Decisões: avalia todos os desvios (, , ternários).

Cenários: expõe quando um ignora o caminho .

Bordas: garante que exceções não fiquem invisíveis.



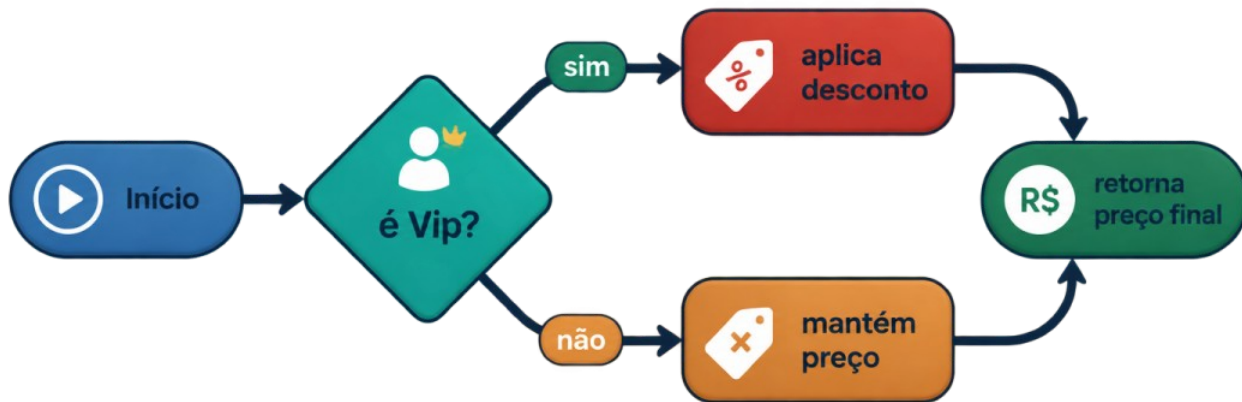
Ponto-chave

100% Line Coverage pode esconder 50% de Branch Coverage se caminhos alternativos forem ignorados.



Exemplo Prático: Branch Oculto

Considere a função de cálculo de desconto abaixo executada com :



O teste executou as linhas e a função, mas o caminho nunca foi acionado. A métrica de branch revelará 50% de cobertura no bloco condicional.

Checagem de Entendimento

Se um arquivo possui 100% de Line Coverage, todos os caminhos condicionais foram necessariamente testados.



 *Prepare-se para explicar o seu raciocínio.*

Checagem de Entendimento



Se um arquivo possui 100% de Line Coverage, todos os caminhos condicionais foram necessariamente testados.

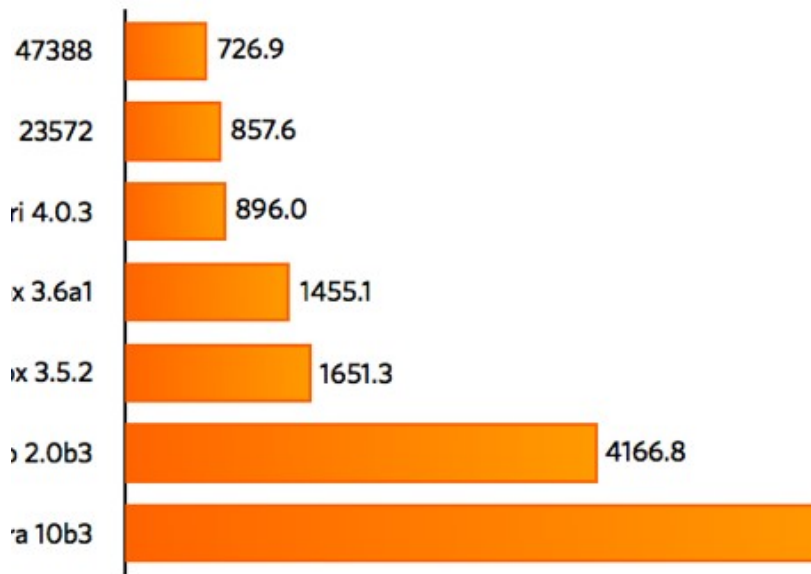


Por que é isso?

Condicionais podem ter branches alternativos ou ternários na mesma linha física que nunca foram ativados.

Provedores: v8 vs Istanbul

Top 6 browser JavaScript performance
providers



Three runs of Sunspider 0.9 JavaScript benchmark
are better

FERRAMENTAS

Motores de Cobertura

Dois motores dominam a análise:

v8: usa instrumentação nativa do Node/Chrome, garantindo máxima velocidade.

Istanbul: transforma a AST com compilador próprio, oferecendo controle e suporte maduro.



Lembre-se

Vitest usa como padrão por sua agilidade e precisão.

Configuração no vitest.config.ts

FERRAMENTAS

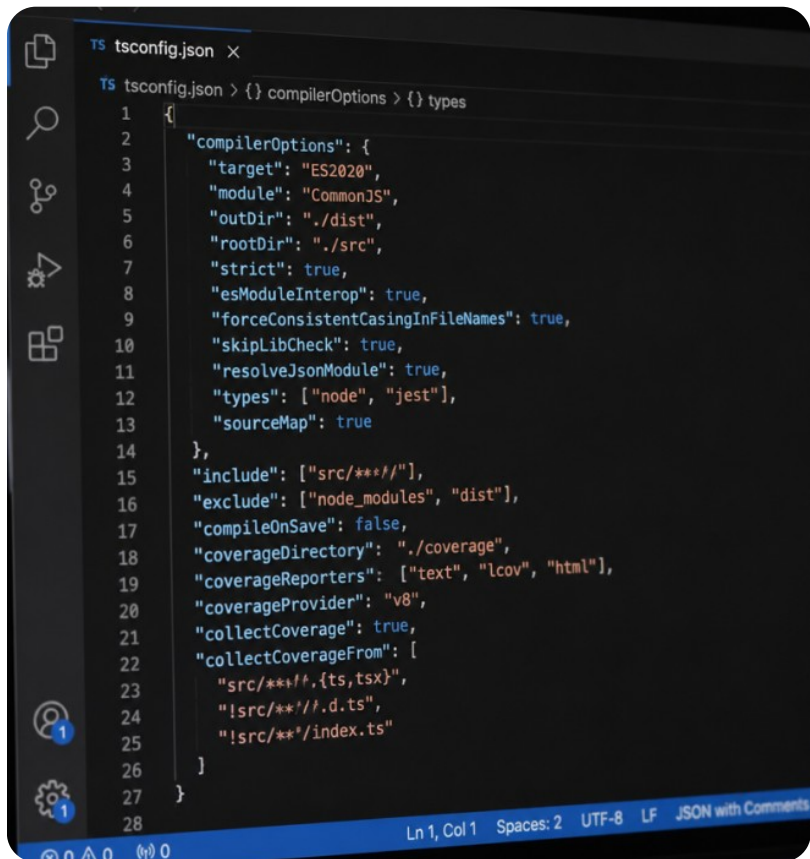
Ativando o Relatório

Para habilitar o gerador de cobertura no Vitest, ajustamos o bloco no arquivo de configuração:

: define entre ou .

: escolhe os formatos como , e .

e : delimitam quais arquivos de produção devem ser auditados.



```
1 {
2   "compilerOptions": {
3     "target": "ES2020",
4     "module": "CommonJS",
5     "outDir": "./dist",
6     "rootDir": "./src",
7     "strict": true,
8     "esModuleInterop": true,
9     "forceConsistentCasingInFileNames": true,
10    "skipLibCheck": true,
11    "resolveJsonModule": true,
12    "types": ["node", "jest"],
13    "sourceMap": true
14  },
15  "include": ["src/**/*.ts"],
16  "exclude": ["node_modules", "dist"],
17  "compileOnSave": false,
18  "coverageDirectory": "./coverage",
19  "coverageReporters": ["text", "lcov", "html"],
20  "coverageProvider": "v8",
21  "collectCoverage": true,
22  "collectCoverageFrom": [
23    "src/**/*.ts",
24    "!src/**/*.d.ts",
25    "!src/**/*.index.ts"
26  ]
27 }
28
```

Estrutura do Arquivo de Configuração

Veja como estruturar o arquivo de configuração para gerar relatórios detalhados no terminal e em HTML:

Parâmetros de Entrada

O campo `instrui` instrui o Vitest a imprimir uma tabela no terminal e exportar uma pasta navegável para inspeção detalhada.

Filtro de Arquivos

Use `para ignorar` , `mocks` e arquivos de `setup`, evitando que código utilitário distorça o percentual real da aplicação.

Press n or j to go to the next uncovered block, b, p or k for the previous block.

File	Statements	Branches	Functions	Lines
src	70.97% 220/310	58.62% 68/116	76.19% 32/42	70.97% 220/310
src/__tests__	100% 26/26	100% 0/0	100% 0/0	100% 26/26
src/components	81.25% 65/80	66.67% 16/24	87.5% 14/16	81.25% 65/80
src/components/common	90.48% 38/42	75% 9/12	100% 6/6	90.48% 38/42
src/components/layout	60.87% 14/23	50% 5/10	50% 2/4	60.87% 14/23

Leitura do Relatório HTML

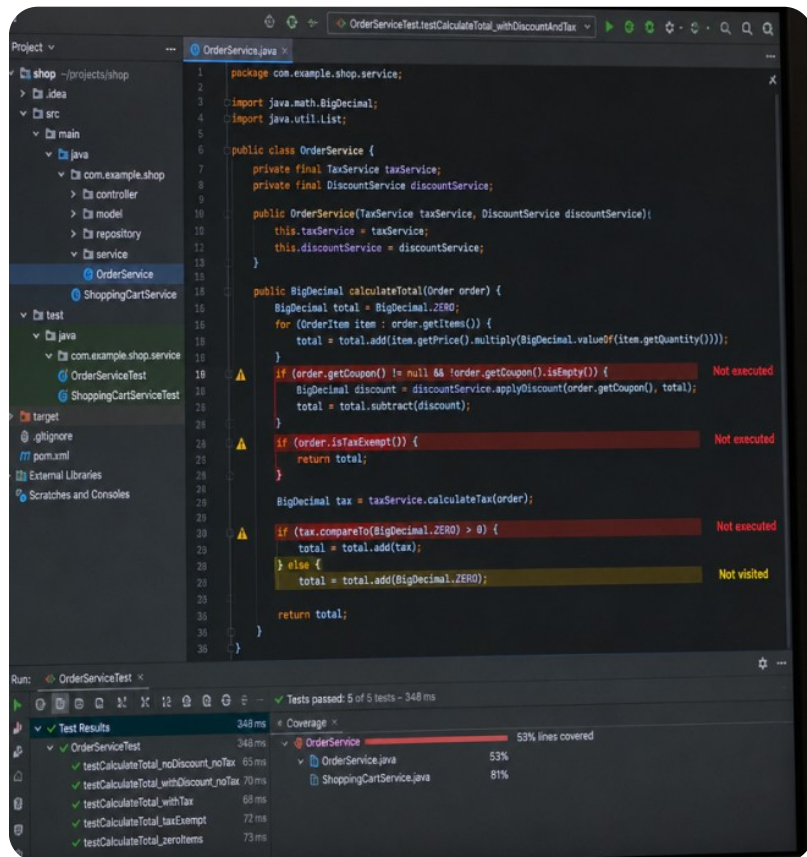
O relatório HTML do Istanbul e v8 gera uma visão navegável completa do projeto, permitindo auditar pastas e arquivos individuais com cores de status intuitivas.



Exemplo

Verde indica cobertura superior a 80%, amarelo alerta entre 50% e 79%, e vermelho marca regiões com cobertura crítica inferior a 50%.

Decifrando Linhas e Marcadores



Destaques no Código

Ao abrir um arquivo específico no relatório HTML:

Contador na margem: exibe quantas vezes cada linha foi executada durante os testes.

Fundo vermelho: destaca linhas que nunca foram visitadas por nenhum cenário.

Letra 'I' ou 'E' (amarelo): alerta de branch não percorrido em condicionais ou .

Checagem: Leitura de Relatório

No relatório HTML gerado pelo Vitest, o que significa um marcador amarelo com a letra 'I' ao lado de uma estrutura condicional?

1. O branch 'if' específico não foi percorrido pelo conjunto de testes.
2. A instrução foi completamente ignorada pelo compilador.
3. A linha foi executada mais de mil vezes na suíte.
4. O arquivo contém um erro fatal de sintaxe JavaScript.

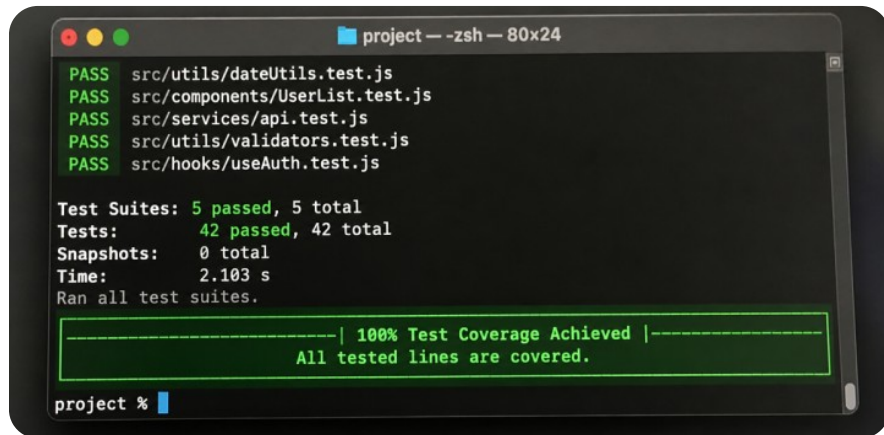
Checagem: Leitura de Relatório



No relatório HTML gerado pelo Vitest, o que significa um marcador amarelo com a letra 'I' ao lado de uma estrutura condicional?

1. O branch 'if' específico não foi percorrido pelo conjunto de testes.
2. A instrução foi completamente ignorada pelo compilador.
3. A linha foi executada mais de mil vezes na suíte.
4. O arquivo contém um erro fatal de sintaxe JavaScript.

A Falsa Sensação dos 100%



```
project — zsh — 80x24

PASS src/utils/dateUtils.test.js
PASS src/components/UserList.test.js
PASS src/services/api.test.js
PASS src/utils/validators.test.js
PASS src/hooks/useAuth.test.js

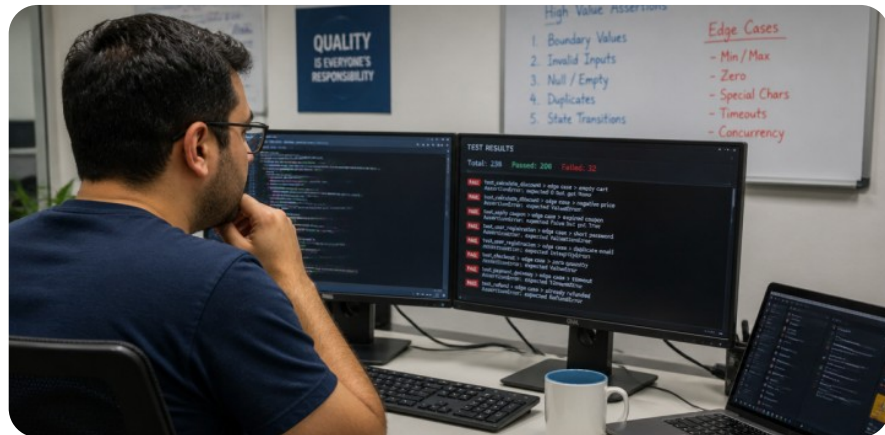
Test Suites: 5 passed, 5 total
Tests: 42 passed, 42 total
Snapshots: 0 total
Time: 2.103 s
Ran all test suites.

-----| 100% Test Coverage Achieved |-----
All tested lines are covered.

project %
```

Cobertura Superficial

Uma suíte pode executar todas as linhas invocando funções sem nenhuma asserção rigorosa, gerando 100% de cobertura vazia.



Qualidade das Asserções

O que protege o software são verificações de regras de negócio, dados malformados e respostas assíncronas inconsistentes.

O Retorno sobre Investimento (ROI)

ROI e Rendimentos Decrescentes

0-70%: Alto retorno; previne bugs críticos.

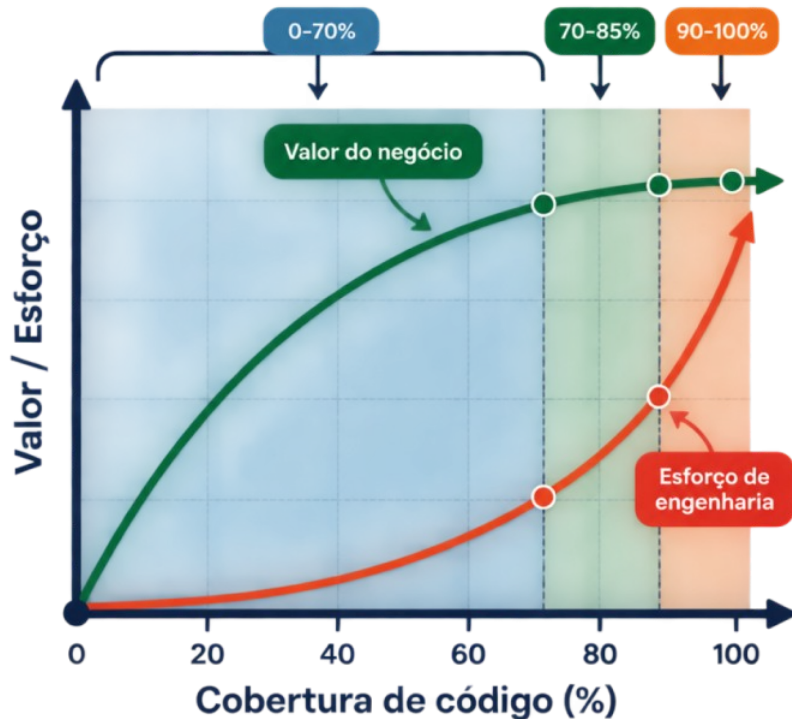
70-85%: Equilíbrio ideal entre esforço e segurança.

90-100%: Custo elevado; foca em código trivial/boilerplate.



Atenção

Exigir 100% gera testes frágeis apenas para bater meta.



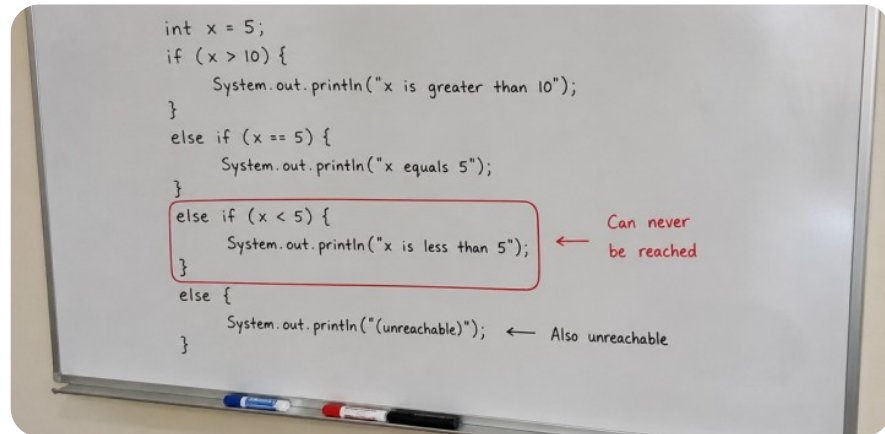
Identificando Código Morto

Relatórios de cobertura são cruciais para higienizar e refatorar o código:



Funções Órfãs

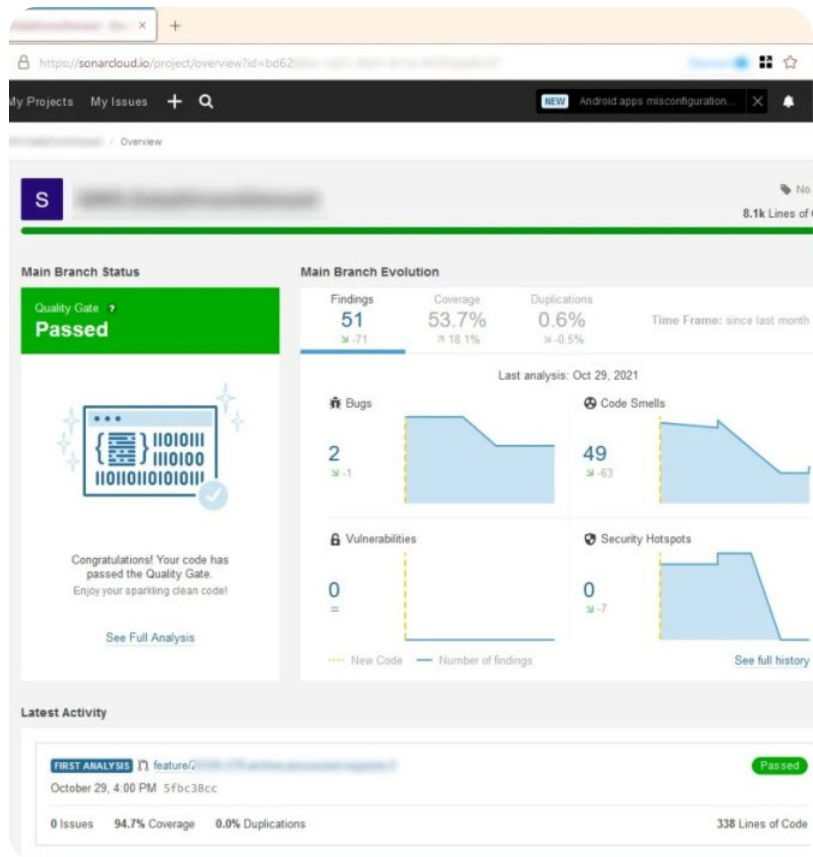
Helpers não chamados por componentes, esquecidos no repositório.



Branches Inalcançáveis

Condicionais redundantes que nunca são atingidas por validações prévias.

Ecossistema: Codecov e SonarQube



INTEGRAÇÃO

Governança Contínua

Em times profissionais, relatórios locais são exportados para serviços em nuvem:

Codecov: comenta automaticamente em Pull Requests mostrando o impacto de novas linhas no percentual global.

SonarQube: avalia cobertura combinada com segurança, duplicações de código e vulnerabilidades (Quality Gate).

Definindo Metas no Pipeline CI

INTEGRAÇÃO

Bloqueio de Regressões

O Vitest permite definir thresholds obrigatórios no pipeline de CI:

Trava de build: a esteira falha se a cobertura de linhas ou branches cair abaixo da meta.

Evolução incremental: exige que código novo (*patch coverage*) tenha meta superior à média antiga do sistema.

Segurança de deploy: impede que código sem testes chegue a produção.



Configuração de Thresholds

No arquivo , estabelecemos metas que quebram o processo caso não sejam atingidas:

Metas Globais

Configurar e garante que o repositório como um todo mantenha um padrão mínimo de qualidade estabelecido pela engenharia.

Rigor por Módulo

Pastas críticas como checkout e pagamentos podem ter limites rigorosos de 90%, enquanto componentes puramente visuais recebem metas flexíveis.

Vídeo: Code Coverage na Prática



Geração de relatórios (Vitest/v8) e análise de branches.

Atividade 1: Mapeando Gaps

Contexto do Exercício

Você recebeu uma aplicação de carrinho de compras que possui apenas 60% de cobertura nos testes unitários.

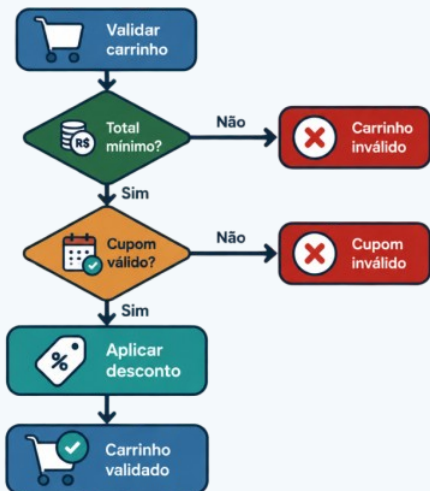
Executar: rodar no terminal.

Abrir: inspecionar o arquivo gerado na pasta .

Identificar: mapear quais funções e blocos condicionais estão marcados em vermelho.

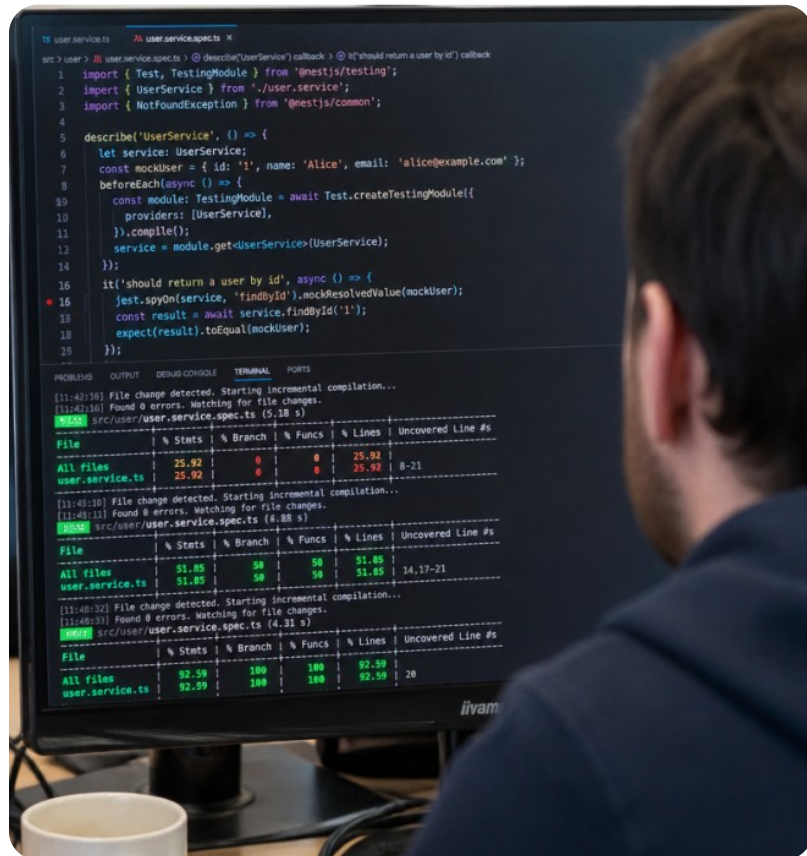


Prática 1: Diagnóstico de Gaps



Abra o terminal do projeto no repositório de exercícios do SENAI e execute o comando de cobertura. Navegue até o módulo 'services/cart.ts'. Localize as 3 linhas não cobertas na função de cálculo de cupom de desconto e anote: qual condição lógica específica foi ignorada pela suíte atual de testes?

Atividade 2: Alçando a 85%



Estratégia de Refatoração

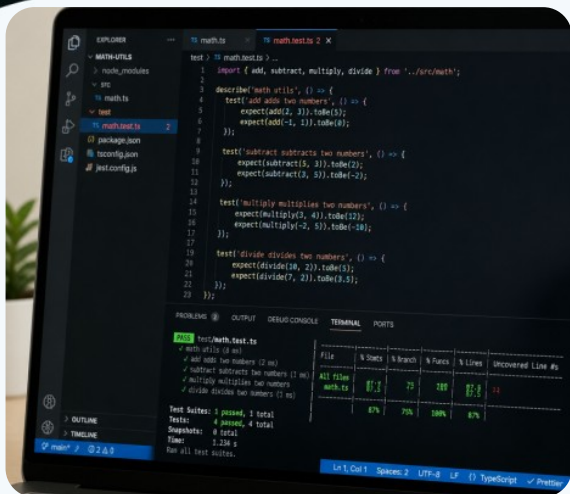
Agora que os pontos cegos foram mapeados, é hora de expandir a suíte:

Sem testes redundantes: evite escrever testes adicionais para fluxos que já estão em 100%.

Foco em caminhos falsos: cubra os branches de erro e exceções no cálculo de cupom.

Meta da atividade: elevar a cobertura do módulo de 60% para 85%.

Prática 2: Refatoração da Suíte



No arquivo 'test/math.test.ts', adicione dois novos blocos 'it()' testando: 1) Tentativa de aplicar cupom expirado; 2) Tentativa de aplicar cupom com valor total inferior ao mínimo exigido. Reexecute o comando com coverage e confirme que o módulo ultrapassou os 85% de Statement e Branch Coverage.

Boas Práticas e Recomendações

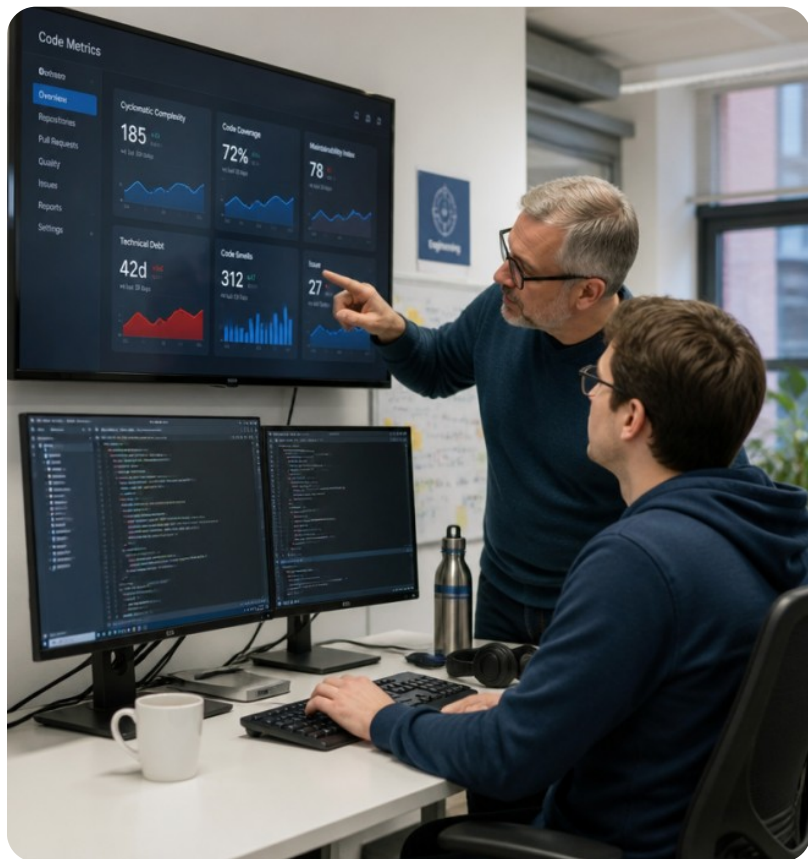
Guia do Desenvolvedor

Para aplicar métricas de cobertura com eficácia na sua carreira:

Use coverage como guia: use o relatório para descobrir o que você esqueceu, não para inflar números.

Priorize regras críticas: módulos financeiros e de autenticação exigem cobertura de branches quase total.

Evite testes sem asserção: todo teste deve validar saídas ou comportamentos esperados.



Desafio: O Ciclo de Cobertura

Ordene os passos profissionais para auditar e aprimorar a cobertura de um módulo de frontend:

Inspeccionar o relatório visual identificando branches não cobertos

Executar o Vitest com o flag de cobertura habilitando relatórios HTML

Escrever novos casos de teste unitários focados nas condições não exercitadas

Verificar se o threshold configurado no pipeline de CI foi atingido

Desafio: O Ciclo de Cobertura



Ordene os passos profissionais para auditar e aprimorar a cobertura de um módulo de frontend:

1
Executar o Vitest com o flag de cobertura habilitando relatórios HTML

2
Inspeccionar o relatório visual identificando branches não cobertos

3
Escrever novos casos de teste unitários focados nas condições não exercitadas

4
Verificar se o threshold configurado no pipeline de CI foi atingido

Próxima Aula: Debugging e TDD

Conectando com o Bloco 08

Vamos inverter a ordem de desenvolvimento:

Red-Green-Refactor: teste antes do código.

TDD: código testável desde o início.

Debugging: breakpoints e isolamento.



Lembre-se

No TDD, a alta cobertura surge naturalmente do processo orientado por testes.

